FDE MODULE 2 | CLASS 4 | INSTRUCTOR SCRIPT

When the Data Fights Back

Making the Workbench survive bad rows, missing files, failing services and reruns, without losing a record or inventing an answer.

Class at a glance

Duration	120 minutes. Quizzes and image moments are inside the section minutes.
Class purpose	Real data is messy. Today the Workbench learns to survive a bad row, a broken business rule, a missing file and a failing outside service. Each gets a planned outcome, every row is counted, and a rerun never repeats work.
Learner outcome	Learners can use `try`, `except`, `else` and `finally`; write their own exception classes and `raise ... from`; read fields without guessing; quarantine bad rows with a reason; prove every row is counted; give batches stable IDs and runs attempt IDs; retry within a limit; mark a failed model answer UNAVAILABLE; and test it all with pytest.
Starting point	The finished Class 3 repository (26 passing checks, recorded mode, no API key needed), plus the Class 4 starter pack.
Business anchor	The ops lead asks: “Last night’s batch finished with no error. Did every case get checked, and can I trust the number?” By the end of class, learners can prove the answer.
Deliverable	A repository where every row is accepted or quarantined with a reason, a bad batch halts before any model call, a rerun repeats no work, retries are bounded and every model answer has a status. 46 passing checks and one incident-style update. Committed and pushed.
Product-level improvement	Every source record is accounted for. Failures have controlled outcomes. Reruns do not duplicate business work.
FDE communication	An incident-style update: the facts, the affected counts, the impact on people and the action needed.
Not in this class	No database, queue, scheduler or dashboard. No real outages: timeouts and rate limits are simulated with fake callers. The tenacity library is compared, not used.
Tools	VS Code, a terminal, Python 3.11, Git and GitHub. An API key is optional.

Class timeline

Quiz time and image moments are inside the section minutes. The clock column is the target time at the end of each section.

Clock	Min	Section	Learners leave with
0-5	5	1. Welcome: the run that lied	A run can finish with no error and still lose records
5-20	15	2. Four kinds of failure and the exception words	try, except, else, finally; own exception classes; raise from. Quiz 1
20-38	18	3. Defensive parsing and quarantine	Parsers that refuse to guess; bad rows set aside with a reason. Quiz 2
38-50	12	4. Required files, the quality gate and reconciliation	A halt before any model call; every row counted. Quiz 3
50-67	17	5. Batch ID, attempt ID, duplicates and safe reruns	A rerun that repeats no finished work. Quiz 4
67-81	14	6. Bounded retry: backoff, jitter and tenacity	Retry only what can succeed later, with growing random waits. Quiz 5
81-98	17	7. Model failures, statuses, the cache and the golden set	UNAVAILABLE instead of an invented answer; two new golden cases. Quiz 6
98-110	12	8. pytest checks for the failure paths	Eight named checks, all green
110-120	10	9. The incident update and close	An update with facts, counts, impact and action; work pushed

> **Instructor note: why so much is pasted**
>
> Every code block is marked Type live or Paste and read. Short, important pieces are typed together so learners feel each line. Long files are pasted from the Final directory and read together. The ideas live in the short examples, not in typing out every file.

Instructor prep checklist

- Build the final repository from the Final directory section. Run `python -m pytest -q` (expect `46 passed`), `python -m mypy` (expect `Success: no issues found in 27 source files`) and `python -m ruff check .` (expect `All checks passed!`).
- Build the starter pack from the Starter files section and run `python -m pytest -q`. Expect `8 failed, 26 passed`. The eight failures are the TODO stubs learners finish in Section 8.
- Rehearse the halted run from Section 4. Expect `HALTED: cases.csv: 4 of 8 rows (50%) were quarantined; the limit is 5%` and exit code 2.
- Run once with `MERIDIAN_AI_MODE=live` and no key. Each suggestion should read `UNAVAILABLE - MERIDIAN_LLM_API_KEY is not set`. No internet is needed.
- Delete the `outputs` folder in your demo copy just before class. Otherwise your first run reuses an old batch, and the attempts log will not match the script.

Starter files

Everything learners need before Section 1. Part A is what they already have from Class 3. Part B is the starter pack. Part C lists the Class 3 files that change during the session.

Download the Starter Files from the provided link.

Starter File Directory Link: Starter Files for Class 4
Starter Files Folder Structure: Starter Files Folder Structure

Part A: what learners already have from Class 3

The finished Class 3 repository, unchanged. `python -m meridian_workbench` prints `4 of 7 cases need attention`, and `python -m pytest -q` prints `26 passed`. A learner whose copy is broken can replace it with the Class 3 final directory.

Part B: the starter pack (new files to copy in before class)

Eight files. Copy each into the same folder in the learner's repository. The Class 3 program still runs; only the eight stub tests fail, on purpose, until Section 8.

File	What it is
data/inventory.csv	Replaces the Class 3 file. Adds a `unit_value` column (amounts such as 19.90) for the amount parser in Section 3.
fixtures/cases_failures.csv	Eight rows with four planted faults. Used in Sections 1, 3 and 4.
fixtures/recorded_responses.json	Replaces the Class 3 file. Adds recorded answers for the two new golden notes.
golden/extra_cases.json	Two new golden cases: G-11 (ambiguous) and G-12 (not enough information). Section 7.
tests/conftest.py	Two ready-made test helpers: `settings_for` and `data_dir`. Read in Section 8.
tests/test_failures.py	Seven test stubs, each with a TODO. They fail until Section 8.
tests/test_rerun.py	One test stub for a safe rerun. It fails until Section 8.
docs/incident_update_template.md	The headings for the incident-style update in Section 9.

> **Instructor note: what the failure fixture is**
>
> `fixtures/cases_failures.csv` is made up by the course team. It has eight rows and four planted faults: an unknown status (RETURNED), a day-first date (07/03/2025), a date that does not exist (2025-02-30) and the case ID C-3004 twice. Every quarantine and halt in this class comes from this one file, so the numbers in this script are the numbers learners will see.

Part C: Class 3 files that change during the session

File	What changes	Section
errors.py (new)	Named intake errors	2
parsing.py (new)	Four careful parsers	3
domain.py	`RejectCategory`, `Ledger`, `SuggestionStatus`; a rejected record gets a category	3, 7
loaders.py	Every row becomes a case or a quarantined record	3
reconcile.py (new)	Required files, duplicates, quality gate, reconciliation	4, 5
batch.py (new)	Batch ID, attempt ID, safe writes, quarantine file, attempt log	5
retry.py (new)	Bounded retry with backoff and jitter	6
llm_client.py, ai_client.py	Own error names, statuses, the cache, no invented fallback	6, 7
pipeline.py, __main__.py	Gates in order, batch folders, safe rerun, exit code 2 on a halt	4, 5
config.py, config/settings.toml, .env.example	`[intake]` and `[retry]` settings; `MERIDIAN_DATA_DIR`	4, 6
results.py, golden.py	The status is written and printed	7
golden/golden_set.json, scoring_baseline.csv	10 cases become 12	7
tests/	Class 3 checks updated; new parsing, retry, failure and rerun checks	3, 6, 8
README.md, docs/terminology.md, pyproject.toml	New run guide, statuses and words; version 0.3.0	9

Final directory

The repository at the end of the class. Every file was run end to end in a clean Python 3.11 environment: pytest (46 passed), mypy (no issues in 27 source files) and ruff (all checks passed).

Download the final directory from the provided link. Use it if a learner falls behind, and for the final check.

Final Files Directory Link: Final Directory after Class 4
Final Directory Folder Structure: Final Directory Structure

Unchanged from Class 3: `.gitignore`, `requirements.txt`, `data/cases.csv`, `data/notes.json`, `data/shipments.csv` and `fixtures/cases_bad_rows.csv`. In `rules.py`, `context.py` and `__init__.py` only blank-line spacing changed, because ruff tidied it. Every other change is listed in Part C above.

Section 1: Welcome: the run that lied

Duration	5 minutes (minute 0 to 5)
Teaching objective	Learners feel the problem before any new tool appears: a run can end without an error and still lose records, keep a duplicate, or hide what it skipped.

Say

Last Monday the batch printed 4 of 7 cases need attention, and we believed it. Nobody asked whether all seven cases were really read, or what happened to a row the program could not understand. A crash is loud, so someone fixes it. A quiet mistake gets believed. Today we make the quiet mistakes loud.

Real-world example: the attendance register

A teacher reads out 40 names. One attendance card has slipped under the desk, so the register says 39 present and nobody notices that a student is missing. The register looks complete. Nothing in it says “one card was not read”. Our batch has the same weakness.

Do: the quiet failure (2 minutes)

Make a scratch copy of the starter folder, swap in the failure fixture as `cases.csv`, and run the Class 3 program. Ask learners to predict first: how many rows will be rejected, and will the run finish?

Paste and run (macOS or Linux; in PowerShell use Copy-Item)

```
cp -r starter scratch
cp starter/fixtures/cases_failures.csv scratch/data/cases.csv
cd scratch
python -m meridian_workbench
```

What Python prints

```
REJECTED row 3 of cases.csv: 'RETURNED' is not a valid OrderStatus
REJECTED row 4 of cases.csv: Invalid isoformat string: '07/03/2025'
REJECTED row 7 of cases.csv: day is out of range for month
Traceback (most recent call last):
  File "<frozen runpy>", line 198, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
  File ".../scratch_starter/src/meridian_workbench/__main__.py", line 6, in <module>
    main()
  File ".../scratch_starter/src/meridian_workbench/pipeline.py", line 42, in main
    results = run(settings)
              ^^^^^^^^^^^^
  File ".../scratch_starter/src/meridian_workbench/pipeline.py", line 36, in run
    assert len({r["case_id"] for r in results}) == len(results)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError
```

What we see

- Three rows were rejected with raw Python messages such as `day is out of range for month`. A support agent cannot act on that.
- Two rows share the case ID C-3004, and the loader accepted both.
- The program then stopped on a bare `AssertionError` with no message, and wrote no report at all.

Pause and predict

Ask: “Which line tells us the second C-3004 row was a duplicate?” None does. The only duplicate check is one `assert` at the very end, and it has no message.

Today we keep three promises. Every section builds one of them.

- **Every source row is accounted for.** It is in the report, or in a quarantine file with a reason.

- **Every failure has a controlled outcome.** A row is set aside, a run halts, or a model answer is marked UNAVAILABLE.
- **A rerun does not repeat business work.** A payment, an email or a model call happens once.

Check for understanding

Ask: Did the Class 3 program say anything useful about the second C-3004 row?

Real-world example: A register that lists the same student twice because two cards carry the same roll number.

Answer: No. It accepted both rows and failed at the end with an empty AssertionError. A check that runs after all the work cannot tell anyone which row to fix.

Transition

To keep these promises we first need names for the different kinds of failure, and a way to react to each one. Python gives us four small words for that.

Section 2: Four kinds of failure and the exception words

Duration	15 minutes (minute 5 to 20), including Quiz 1
Teaching objective	Learners sort a failure into one of four kinds, choose the outcome for each, and use `try`, `except`, `else`, `finally`, their own exception classes and `raise ... from`.

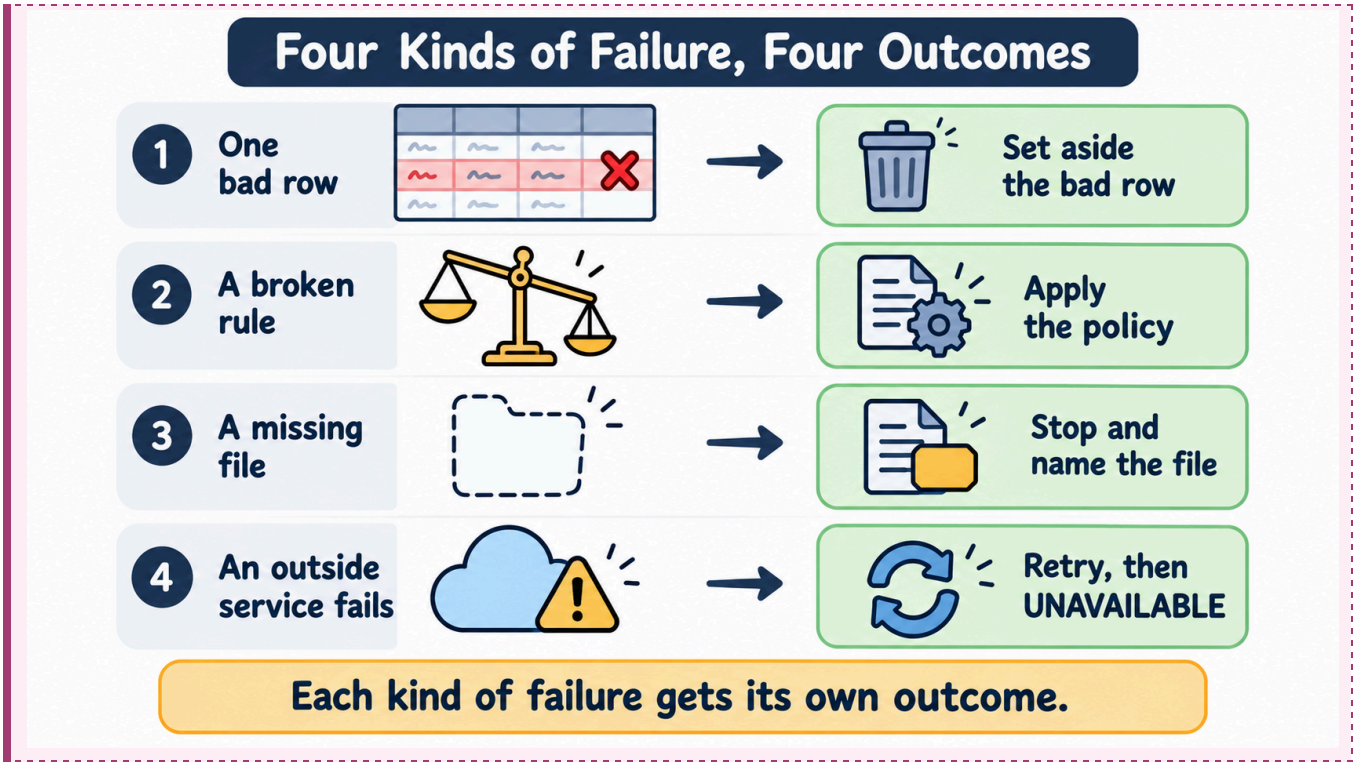
Say

A problem with data is not one thing. If we treat every problem the same way, we either stop too often or hide too much. So we sort problems into four kinds, and each kind gets its own outcome.

Kind	Meridian example	Planned outcome
Malformed record	A date written as 07/03/2025, or a status of RETURNED	Set that one row aside. The batch continues.
Business-rule failure	Case C-3004 appears on two rows; the rule is one row per case	Apply the declared duplicate policy: set the repeat aside, or stop the batch.
Missing or unreadable source	shipments.csv did not arrive, or notes.json is broken JSON	Stop before any work, and name the file.
Outside service fails	The model times out, or says 429 Too Many Requests	Retry a few times, then mark it UNAVAILABLE. The facts stay.

Real-world example: one delivery van, four kinds of trouble

A van leaves with 30 parcels. One label is torn: that parcel is put aside and the other 29 go out. One parcel is addressed outside the delivery area: the label is readable, but it breaks a rule, so it goes back with a note. If the van never reaches the depot, nobody starts the round. If a customer's phone is busy, the driver tries again a little later, and after a few tries writes "not reachable". He never writes "delivered" to fill the gap.



Step 1: the four words (type live)

Create `scratch_keywords.py` in the project root, type this and run it with `python scratch_keywords.py`.

scratch_keywords.py

```
from datetime import date


def read_promised(raw: str) -> date:
    return date.fromisoformat(raw)


def check_row(row: dict[str, str]) -> str:
    try:
        promised = read_promised(row["promised_date"])
    except ValueError:
        return "quarantined: promised_date is not a date"
    else:
        return f"accepted, due {promised.isoformat()}"
    finally:
        print(f"checked {row['case_id']}")


print(check_row({"case_id": "C-3003", "promised_date": "07/03/2025"}))
print(check_row({"case_id": "C-3001", "promised_date": "2025-03-07"}))
```

What Python prints

```
checked C-3003
quarantined: promised_date is not a date
checked C-3001
accepted, due 2025-03-07
```

What each piece does

- **try** holds the line that might fail: reading the date.
- **except ValueError** runs only if that line raised a `ValueError`. Here it returns the quarantine message.

- **else** runs only if the `try` block finished without an error. The success code lives here, not inside `try`.
- **finally** runs every time, success or failure. It printed `checked ...` before each answer came back.

Analogy to use

Making tea. Try: boil the water. Except: if the gas runs out, use the kettle. Else: if nothing went wrong, add the tea leaves. Finally: switch off the stove, every single time, whatever happened.

Pause and predict

Before running, ask: "For C-3003 the date fails. Which line prints first?" `checked C-3003` prints first, because `finally` runs before the function hands back its answer.

Step 2: give each kind a name (type live)

Python's `ValueError` says a value was bad, but not which part of our program cares. Our own exception classes give each kind of problem a name. Create `src/meridian_workbench/errors.py`.

src/meridian_workbench/errors.py

```
"""The intake vocabulary. Every problem we expect has a name, and each name has one outcome."""


class IntakeError(Exception):
    """Base class for every problem we expect while taking in a batch."""


class MalformedRecord(IntakeError):
    """One row (or one note) cannot be read. Only that record is quarantined."""


class RuleFailure(IntakeError):
    """A readable record or the whole batch breaks a business rule."""


class MissingSource(IntakeError):
    """A required file is absent. The whole run stops before any work starts."""


class UnreadableSource(IntakeError):
    """A required file exists but cannot be read as a whole, for example broken JSON."""


class QualityGateFailed(IntakeError):
    """The batch is not good enough to use. The run halts before any model call."""


class ReconciliationError(IntakeError):
    """Accepted plus quarantined does not equal the input. This is a bug in our code."""
```

What each piece does

- `class MalformedRecord(IntakeError):` creates a new kind of error that belongs to the `IntakeError` family.
- The docstring is the whole body. A new exception class needs no code; its name is the point.
- The family lets a caller catch one kind exactly (`except MalformedRecord`) or every intake problem at once (`except IntakeError`).

Step 3: keep the original cause with raise ... from (paste and read)

Paste `src/meridian_workbench/parsing.py` from the Final directory. Read `parse_date` together. It turns Python's low-level error into our own name, and keeps the original error attached.

src/meridian_workbench/parsing.py (parse_date)

```
def parse_date(raw: str | None, field: str = "promised_date") -> date:
    """Only YYYY-MM-DD is accepted. 07/03/2025 could mean 7 March or 3 July, so we refuse it."""
```

```
    text = require_text(raw, field)
    if not DATE_SHAPE.match(text):
        raise MalformedRecord(f"{field} {text!r} is not YYYY-MM-DD")
    try:
        return date.fromisoformat(text)
    except ValueError as error:
        raise MalformedRecord(f"{field} {text!r} is not a real calendar date") from error
```

Run in the terminal

```
python -c "from meridian_workbench.parsing import parse_date; parse_date('2025-02-30')"
```

What Python prints

```
Traceback (most recent call last):
  File ".../final/src/meridian_workbench/parsing.py", line 28, in parse_date
    return date.fromisoformat(text)
           ^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: day is out of range for month

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "<string>", line 2, in <module>
  File ".../final/src/meridian_workbench/parsing.py", line 30, in parse_date
    raise MalformedRecord(f"{field} {text!r} is not a real calendar date") from error
meridian_workbench.errors.MalformedRecord: promised_date '2025-02-30' is not a real calendar date
```

Read the traceback from the bottom. The last line is our plain sentence. Above it, Python says the `ValueError` was the direct cause. Without `from error`, Python would say the second error happened "during handling" of the first, which reads like a second bug.

Check for understanding

Ask: What is the difference between a MalformedRecord and a MissingSource?

Real-world example: One torn parcel label (put that parcel aside) versus the van never arriving (nobody starts the round).

Answer: A malformed record affects one row: set it aside and carry on. A missing source affects the whole run: stop before any work and name the file.

Common mistakes to expect

- Writing a bare `except:` or `except Exception:`. It also hides real bugs, such as a typo in a variable name.
- Putting the success code inside `try` instead of `else`, so its own errors get caught by mistake.
- Forgetting `from error` when wrapping an error, which makes the traceback harder to read.
- Writing `return` inside `finally`. It silently throws the error away.

If learners struggle

The idea that must land is "each problem has a name, and the name decides the outcome". If time is short, paste errors.py instead of typing it, and keep the keyword drill.

Quiz 1 of 6 | After Section 2 (minute 20) | Multiple choice (one answer) | about 90 seconds

Checks: When else and finally run.

Question. In check_row, the date is valid. Which blocks run?

A. try, except and else
B. try, except and finally
C. try, else and finally
D. try and finally only

Correct answer: C

Why C is correct: There was no error, so except is skipped. else runs because try succeeded, and finally always runs.

Why the others are wrong: A and B include except, which needs an error to happen. A also leaves out finally, which runs every time. D forgets else, which is exactly where the success code lives.

Instructor tip: Point at the drill output: “checked C-3001” comes first, then “accepted, due 2025-03-07”.

Transition

We can now name a bad row. Next, we read each field carefully, so that we notice a bad value instead of guessing what it meant.

Section 3: Defensive parsing and quarantine

Duration	18 minutes (minute 20 to 38), including Quiz 2
Teaching objective	Learners use parsers that accept only what they can read without guessing, keep the raw value, and set each refused row aside with a category and a reason.

Say

Every field in a file is a claim: when something was promised, how many units there are, where the order is. A parser accepts a claim only when it can read it without guessing. If it would have to guess, it refuses and says why. A guess looks exactly like a fact in a report, and nobody can tell them apart later.

Real-world example: the barcode that will not scan

At a supermarket till, one item’s barcode will not scan. A good cashier does not type in a price that “looks about right”. She puts the item to one side, rings up the rest, and calls a supervisor with the item in her hand. The customer still pays for everything that scanned, and the problem item is not lost.

Step 1: four careful parsers (paste and read)

You pasted `parsing.py` in Section 2. Each parser checks the shape of the text first, then turns it into a real value. The table shows what each one accepts and refuses.

Parser	Accepts	Refuses
require_text	Any non-blank text; spaces at the edges are trimmed	An empty or missing field
parse_date	2025-03-07	07/03/2025, 20250307, 2025-02-30
parse_units	0, 12	-3, 1.5, 1,000
parse_amount	19.90	19.9, 1,250.00, 1.250, NaN
parse_status	SHIPPED (spaces at the edges trimmed)	shipped, RETURNED

src/meridian_workbench/parsing.py (parse_amount)

```
def parse_amount(raw: str | None, field: str = "unit_value") -> Decimal:
    """Money in our data contract: a dot, exactly two decimal places, no thousands separator."""
    text = require_text(raw, field)
    if not AMOUNT_SHAPE.match(text):
        raise MalformedRecord(
            f"{field} {text!r} must look like 19.99 (two decimal places, no commas)"
        )
    try:
        value = Decimal(text)
    except InvalidOperation as error:  # the shape check should catch this first; belt and braces
        raise MalformedRecord(f"{field} {text!r} is not a number") from error
    if not value.is_finite():
        raise MalformedRecord(f"{field} {text!r} is not a finite number")
```

```
    return value
```

`AMOUNT_SHAPE` is a regular expression: a pattern for text. `^` means the start, `$` the end, `\d+` one or more digits, and `\.\d{2}` a dot followed by exactly two digits. Only text with that exact shape reaches `Decimal`.

> **Common gotcha**
> `Decimal("1.250")` is one and a quarter. A supplier that uses a dot for thousands means one thousand two hundred and fifty. Both readings are valid numbers, so no clever code can tell them apart. The fix is a rule: money has a dot for decimals and exactly two decimal places. Anything else is refused.

Step 2: the ledger: the books for one file (paste and read)

Paste `domain.py` from the Final directory. Three pieces are new for intake.

src/meridian_workbench/domain.py (new intake pieces)

```
class RejectCategory(StrEnum):
    MALFORMED = "MALFORMED"  # a row or note we cannot read
    RULE_FAILURE = "RULE_FAILURE"  # readable, but it breaks a business rule
    DUPLICATE = "DUPLICATE"  # a repeat of a key we already kept
    MISSING_SOURCE = "MISSING_SOURCE"

@dataclass(frozen=True)
class RejectedRecord:
    """A quarantined record: where it came from, what it said, and why it was not used."""

    source: str
    row_number: int
    category: RejectCategory
    raw: str
    reason: str

@dataclass
class Ledger:
    """The books for one source file. Every row read is either accepted or quarantined."""

    source: str
    input_rows: int = 0
    accepted: int = 0
    rejected: list[RejectedRecord] = field(default_factory=list)

    def reject(self, row_number: int, category: RejectCategory, raw: str, reason: str) -> None:
        self.rejected.append(RejectedRecord(self.source, row_number, category, raw, reason))
```

What each piece does

- `RejectCategory` is the closed list of reasons a row can be set aside.
- `RejectedRecord` keeps the file, the line number, the category, the reason and the raw row exactly as it was read.
- `Ledger` counts every input row and holds the rejected ones. Section 4 uses it to check the books.

Step 3: every row leaves through one of two doors (paste and read)

Paste `loaders.py` from the Final directory and read `load_cases`.

src/meridian_workbench/loaders.py (load_cases)

```
def load_cases(path: Path) -> tuple[list[tuple[int, SupportCase]], Ledger]:
    ledger = Ledger(path.name)
    rows: list[tuple[int, SupportCase]] = []
    for row_number, row in read_csv_rows(path):
        ledger.input_rows += 1
        try:
            case = SupportCase(
                case_id=require_text(row.get("case_id"), "case_id"),
                order_id=require_text(row.get("order_id"), "order_id"),
                order_status=parse_status(row.get("order_status")),
```

```
                promised_date=parse_date(row.get("promised_date")),
            )
        except MalformedRecord as problem:
            ledger.reject(row_number, RejectCategory.MALFORMED, str(dict(row)), str(problem))
            continue
        rows.append((row_number, case))
    ledger.accepted = len(rows)
    return rows, ledger
```

- `ledger.input_rows += 1` runs first, so even a broken row is counted.
- If any field fails, `MalformedRecord` is caught, the row is quarantined with its line number, and `continue` moves on.
- A good row is kept together with its line number. Section 5 needs it to report duplicates.

Do: run the loader on the failure fixture (3 minutes)

Paste into python, from the project root

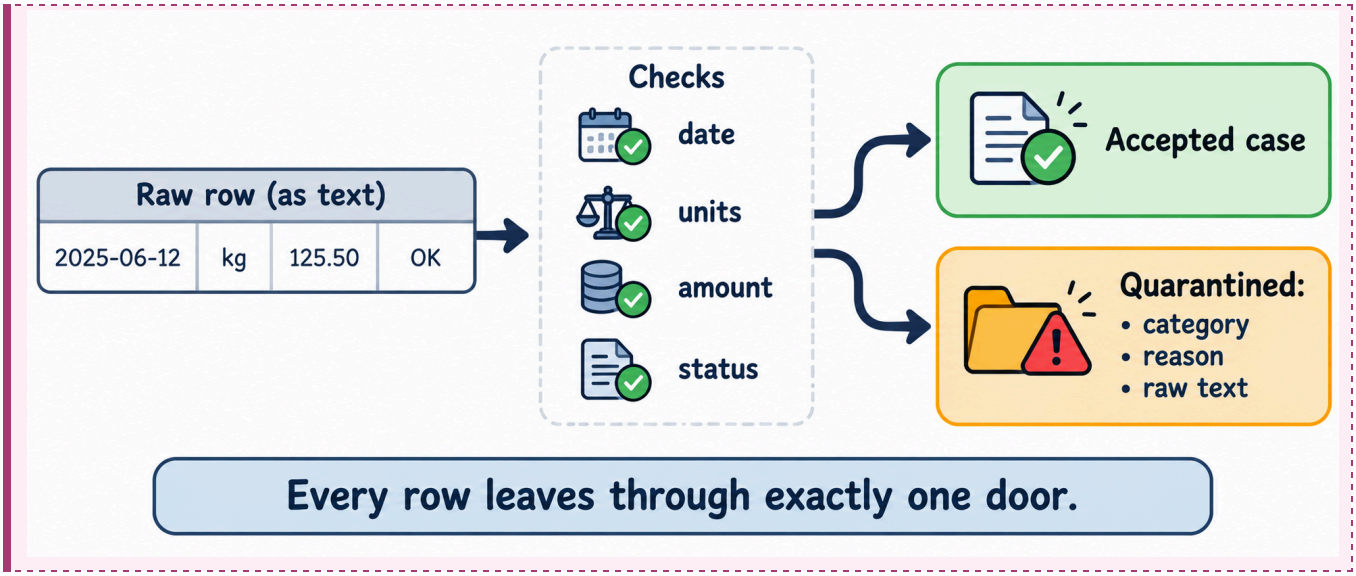
```
from pathlib import Path
from meridian_workbench.loaders import load_cases

rows, ledger = load_cases(Path('fixtures/cases_failures.csv'))
print('input rows:', ledger.input_rows)
print('accepted:', ledger.accepted)
print('quarantined:', len(ledger.rejected))
for r in ledger.rejected:
    print(r.row_number, r.category, r.reason)
```

What Python prints

```
input rows: 8
accepted: 5
quarantined: 3
3 MALFORMED order_status 'RETURNED' is not one of PROCESSING, SHIPPED, DELIVERED, CANCELLED
4 MALFORMED promised_date '07/03/2025' is not YYYY-MM-DD
7 MALFORMED promised_date '2025-02-30' is not a real calendar date
```

Compare this with Section 1. The messages now name the field, the value and the rule. The duplicate row 6 is still accepted here, because a loader reads one row at a time. Section 5 catches it.



Pause and predict

Ask: “Row 3 says RETURNED. Should the program guess that it means CANCELLED?” No. The guess would be counted as a fact, and nobody could tell later. The row is set aside with the reason, and a person decides.

Check for understanding

Ask: A row has a valid date, but its order_id is empty. Which check catches it?

Real-world example: A parcel label with a clear date but no address line. The date is fine; the parcel still cannot be delivered.

Answer: `require_text` on `order_id`. Each field has its own check, so each problem gets its own reason.

Common mistakes to expect

- "Fixing" a value in the parser, such as turning 07/03/2025 into a date. That is a guess.
- Cleaning the raw text before saving it to the quarantine. The quarantine must keep the row exactly as read.
- Counting a row only after it passes, so broken rows disappear from the count.
- Calling `.lower()` or `.upper()` on statuses to be helpful. The data contract says statuses are exact.

If learners struggle

Paste parsing.py and loaders.py rather than typing them. The idea that must land is "every row goes through exactly one door: accepted or quarantined, never lost".

Quiz 2 of 6 | After Section 3 (minute 38) | Multiple choice (one answer) | about 90 seconds

Checks: Why a parser refuses instead of fixing.

Question. parse_date receives the text 07/03/2025. What should happen?

A. Raise MalformedRecord, so the row is quarantined

B. Read it as 7 March, the most common meaning

C. Read it as 3 July, the meaning used in the US

D. Skip the row without leaving any record of it

Correct answer: A

Why A is correct: The text has two possible meanings and the parser cannot know which one was meant. It refuses, keeps the raw text, and a person decides from the quarantine file.

Why the others are wrong: B and C are guesses that would look like facts in the report. D loses a row with no trace, which breaks the first promise that every row is counted.

Instructor tip: Ask the room which date it is before revealing the answer. You will get both answers. That is the point.

Transition

Rows are now accepted or quarantined. But a whole batch can still be wrong: a file can be missing, or half the rows can be bad. Those are decisions about the batch, and they come next.

Section 4: Required files, the quality gate and reconciliation

Duration	12 minutes (minute 38 to 50), including Quiz 3
Teaching objective	Learners stop a batch before any model call when a file is missing or too many rows are bad, and check the books so that no row can disappear quietly.

Say

A batch is four files that must agree. Before we trust any number, we check the batch as a whole, in a fixed order. Are all the files here? Is the batch good enough to use? Do the books balance? Remember: Do not run pytest until Section 8; the old tests no longer match the new code.

Step 1: the intake, in order (paste and read)

Paste these from the Final directory now: `reconcile.py`, `batch.py`, `retry.py`, `llm_client.py`, `ai_client.py`, `results.py`, `pipeline.py`, `config.py`, `__main__.py` and `config/settings.toml`. The new `pipeline.py` uses all of them, so the program runs again only when every one is in place. Sections 5 to 7 read the parts pasted now. Start with `run_intake`: the whole intake on one screen.

src/meridian_workbench/pipeline.py (run_intake)

```
def run_intake(data_dir: Path, settings: Settings, out_dir: Path) -> Intake:
    """Steps in order: files, rows, duplicates, quarantine, gate, books."""
    require_sources(data_dir)
    case_rows, cases_ledger = load_cases(data_dir / "cases.csv")
    notes, notes_ledger = load_notes(data_dir / "notes.json")
    shipments, shipments_ledger = load_shipments(data_dir / "shipments.csv")
    inventory, inventory_ledger = load_inventory(data_dir / "inventory.csv")
    case_rows = apply_duplicate_policy(case_rows, cases_ledger, settings.duplicate_policy)

    ledgers = [cases_ledger, notes_ledger, shipments_ledger, inventory_ledger]
    rejected = [record for ledger in ledgers for record in ledger.rejected]
    write_quarantine(
        rejected, out_dir / "quarantine.csv"
    )  # written before any gate can halt the run
    for ledger in ledgers:
        check_quality(ledger, settings.max_quarantine_share)
        reconcile(ledger)
    return Intake(
        cases=[case for _, case in case_rows],
        notes=notes,
        shipments=shipments,
        inventory=inventory,
        ledgers=ledgers,
        rejected=rejected,
    )
```

- Files are checked first, then read, then the duplicate rule runs.
- The quarantine file is written before any gate can stop the run, so the people fixing the data always get the list.
- Only then do the quality gate and the books run, file by file.

Step 2: every required file is present

src/meridian_workbench/reconcile.py (require_sources)

```
def require_sources(data_dir: Path) -> None:
    """Check every required file before reading any of them. Name all the missing ones at once."""
    missing = [name for name in REQUIRED_SOURCES if not (data_dir / name).is_file()]
    if missing:
        raise MissingSource(f"required source(s) missing from {data_dir}: {', '.join(missing)}")
```

The check runs before any file is read, and it collects every missing name, so one run tells the person everything that is missing.

> **Real-world example: the school admission desk**
>
> A school needs a birth certificate, a photo and an address proof. If two are missing, a good clerk tells the parent about both at once. She does not send them home for one, and then again for the other.

Step 3: the quality gate

src/meridian_workbench/reconcile.py (check_quality)

```
def check_quality(ledger: Ledger, max_share: float) -> None:
    """Halt when too many rows were quarantined, or when the file is empty."""
    if ledger.input_rows == 0:
        raise QualityGateFailed(
            f"{ledger.source} has no data rows; an empty file is a problem, not a quiet success"
        )
```

```
    quarantined = len(ledger.rejected)
    share = quarantined / ledger.input_rows
    if share > max_share:
        raise QualityGateFailed(
            f"{ledger.source}: {quarantined} of {ledger.input_rows} rows ({share:.0%}) were "
            f"quarantined; the limit is {max_share:.0%}"
        )
```

A few bad rows are normal. A large share usually means the file changed shape, or the export broke. The limit is `max_quarantine_share = 0.05` (5 percent) in `config/settings.toml`. Above it, the batch halts. An empty file also halts, because an empty file is a problem, not a quiet success.

> **Real-world example: the sack of rice**
>
> A few small stones in a sack of rice are normal; you pick them out. If half the sack is stones, you do not cook the good half and call dinner a success. You return the whole sack.

Do: the halted run (4 minutes)

Build a scratch data folder that uses the failure fixture as `cases.csv`, and point the program at it. Ask learners to predict: will there be a summary, and what will the exit code be?

Paste and run (macOS or Linux; in PowerShell set $env:MERIDIAN_DATA_DIR and use $LASTEXITCODE)

```
mkdir -p scratch/data
cp data/notes.json data/shipments.csv data/inventory.csv scratch/data/
cp fixtures/cases_failures.csv scratch/data/cases.csv
MERIDIAN_DATA_DIR=scratch/data python -m meridian_workbench
echo $?
```

What the terminal shows

```
HALTED: cases.csv: 4 of 8 rows (50%) were quarantined; the limit is 5%
2
```

`2` is the exit code: the number a program hands back when it ends. A scheduler that sees 2 knows the batch did not finish. Now open the quarantine file in the new batch folder.

outputs/batches/<batch id>/quarantine.csv

```
source,row_number,category,reason,raw
cases.csv,3,MALFORMED,"order_status 'RETURNED' is not one of PROCESSING, SHIPPED, DELIVERED,
CANCELLED","{'case_id': 'C-3002', 'order_id': 'O-7002', 'order_status': 'RETURNED', 'promised_date':
'2025-03-08'}"
cases.csv,4,MALFORMED,promised_date '07/03/2025' is not YYYY-MM-DD,"{'case_id': 'C-3003', 'order_id':
'O-7003', 'order_status': 'SHIPPED', 'promised_date': '07/03/2025'}"
cases.csv,6,DUPLICATE,case_id C-3004 already appears on row 5; first_wins keeps that row,"C-3004,O-7005"
cases.csv,7,MALFORMED,promised_date '2025-02-30' is not a real calendar date,"{'case_id': 'C-3005',
'order_id': 'O-7006', 'order_status': 'SHIPPED', 'promised_date': '2025-02-30'}"
```

Each line names the file, the line number, the category, the reason and the raw text. A person can open `cases.csv`, go to line 3 and fix it. A quarantine is a worklist, not a bin.

> **Pause and predict**
>
> Ask: “The halted batch folder has quarantine.csv and attempts.log, but no summary.json. Why?” The summary is written last, only when the whole run completes. No summary means the batch did not finish.

Step 4: the books balance

src/meridian_workbench/reconcile.py (reconcile)

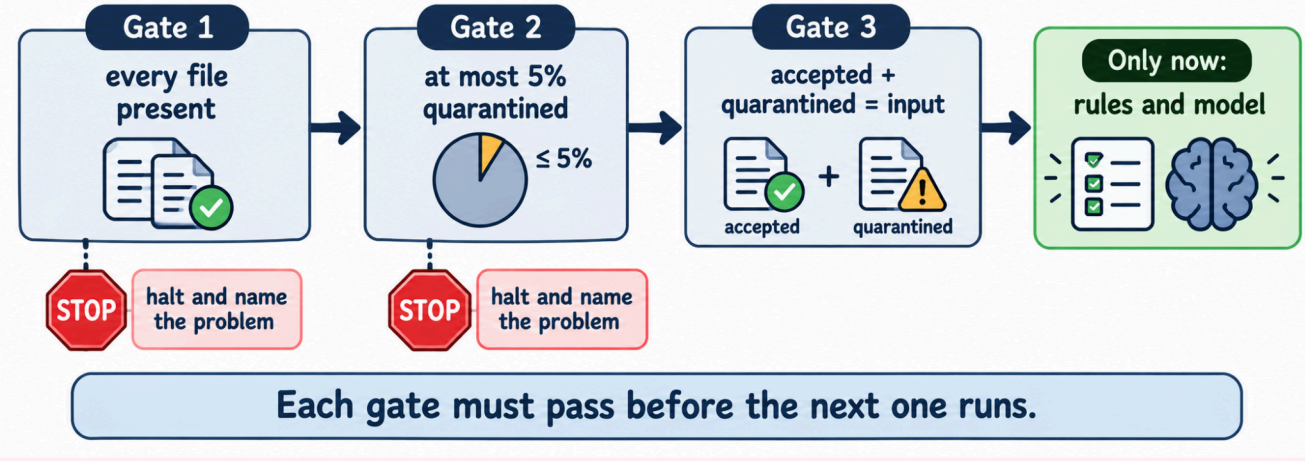
```
def reconcile(ledger: Ledger) -> None:
    """Every input row is accepted or quarantined. Anything else is a bug, and we say so."""
    quarantined = len(ledger.rejected)
    if ledger.accepted + quarantined != ledger.input_rows:
        difference = ledger.input_rows - ledger.accepted - quarantined
```

```
        raise ReconciliationError(
            f"{ledger.source}: {ledger.input_rows} rows in, {ledger.accepted} accepted and "
            f"{quarantined} quarantined; difference of {difference} (not zero)"
        )
```

Every row read must end up accepted or quarantined, so the two must add up to the input. Bad data cannot break this sum. If it fails, our code has a bug, for example a new branch that forgot to count. That is exactly what this check is for.

Analogy to use

The cash drawer at closing time. Opening cash plus sales minus refunds must equal the cash counted. If it does not, the customers did nothing wrong; the counting did.



Check for understanding

Ask: A file has 100 rows: 96 accepted and 4 quarantined. Do the books balance, and does the gate pass?

Real-world example: A class register: 96 present and 4 absent makes 100 on the roll.

Answer: The books balance (96 + 4 = 100). The gate passes too: 4 percent is under the 5 percent limit.

Common mistakes to expect

- Checking each file only when it is read, so half the work is done before the missing file is noticed.
- Writing the quarantine after the gate, so a halted run leaves no list of the bad rows.
- Treating an empty file as a success with zero problems.
- Raising the limit to 100% to make a halt go away. The halt is the system working.

Quiz 3 of 6 | After Section 4 (minute 50) | Multiple choice (one answer) | about 90 seconds

Checks: What the quality gate does with a bad batch.

Question. cases.csv has 8 rows and 4 are quarantined. The limit is 5%. What happens?

A. The 4 good rows are reported and the run passes

B. The file is read again three times to be sure

C. The 4 bad rows are corrected automatically

D. The run halts with HALTED and exits with code 2

Correct answer: D

Why D is correct: 50% is far above 5%, so the batch is not trusted. The run halts before any model call and prints the counts. The quarantine file is still written so a person can fix it.

Why the others are wrong: A publishes a report built on half a file. B cannot help, because reading bad data again gives the same bad data. C is guessing, and the program never fixes data on its own.

Instructor tip: Put the halted output back on screen and read the message aloud, word by word.

Transition

The files are present, the batch is good enough and the books balance. Next, the batch needs a name that stays the same, and a rule that a finished batch is never done twice.

Section 5: Batch ID, attempt ID, duplicates and safe reruns

Duration	17 minutes (minute 50 to 67), including Quiz 4
Teaching objective	Learners give each batch a stable ID and each run an attempt ID, choose a duplicate policy on purpose, and make a rerun return the finished result instead of repeating the work.

Say

A run that finishes once is easy. Trouble starts when a run is started twice, or crashes and is started again. That is how payments get paid twice and emails get sent twice. Today every batch gets a name, every run gets a record, and a finished batch is never done again.

Real-world example: the train booking reference

When you book a train ticket, you get a booking reference (a PNR). Check the status ten times and it is still the same reference, and you still have one seat. Each check is a separate attempt; the booking is one. Our batch ID is the booking reference, and each run is an attempt.

Step 1: the batch ID (read together)

Open `batch.py`, pasted in Section 4. The batch ID is a fingerprint of what the batch is: the date, the rules, and the names and bytes of the four files. It is not a random number.

src/meridian_workbench/batch.py (batch_id_for)

```
def batch_id_for(files: Sequence[Path], batch_date: date, rules_text: str) -> str:
    """The same files, date and rules always give the same ID, on any machine.

    rules_text holds every setting that can change an answer, so a live run never reuses
    recorded answers, and a different duplicate policy never reuses an old result.
    """
    digest = hashlib.sha256()
    digest.update(batch_date.isoformat().encode("utf-8"))
    digest.update(rules_text.encode("utf-8"))
    for path in sorted(
        files, key=lambda p: p.name
    ):  # sorted, so the order of listing cannot matter
        digest.update(path.name.encode("utf-8"))
        digest.update(b"\0")
        digest.update(path.read_bytes())
        digest.update(b"\0")
    return "B-" + digest.hexdigest()[:12]
```

What each piece does

- `hashlib.sha256()` makes a fingerprint: the same input always gives the same long code. We keep the first 12 characters.
- Files are sorted by name, so the order they are listed in cannot change the ID. A copied file has the same bytes, so it gets the same ID.
- `rules_text` holds every setting that can change an answer: AI mode, model, prompt version, duplicate policy and quarantine limit.

The rules part fixed a real mistake made while building this class: the first version hashed only the files and the date, so a live run reused the recorded run's answers. Now the mode is in the fingerprint.

The same files, two AI modes

```
recorded B-1bf9b12a0999
live B-5ab2e0c27bc3
```

Step 2: the attempt ID and the attempt log

Each run also gets an attempt ID: a random UUID that changes every time, even when the batch ID stays the same. Every run that completes, halts or is reused adds one line to `attempts.log`: the time, the attempt ID and the outcome. The log is only ever added to, never rewritten.

Step 3: write files safely

src/meridian_workbench/batch.py (write_json_atomic)

```
def write_json_atomic(path: Path, data: Any) -> None:
    """Write to a temporary name, then rename. A reader never sees a half-written file."""
    path.parent.mkdir(parents=True, exist_ok=True)
    temp = path.with_name(path.name + ".tmp")
    temp.write_text(json.dumps(data, indent=2, sort_keys=True), encoding="utf-8")
    os.replace(temp, path)
```

A power cut halfway through writing leaves half a file, and the next reader sees broken JSON. So we write to a temporary name and then rename it in one step with `os.replace`. A reader sees the old file or the new file, never half of one.

> **Analogy to use**
>
> A shop changing its price board. It writes the new prices on a fresh board, then swaps the boards in one move. It does not rub out the old prices while customers are reading them.

Step 4: duplicates: decide the rule before the run

src/meridian_workbench/reconcile.py (apply_duplicate_policy)

```
def apply_duplicate_policy(
    rows: list[tuple[int, SupportCase]], ledger: Ledger, policy: str
) -> list[tuple[int, SupportCase]]:
    """A case_id must appear once. first_wins keeps the first row; reject_batch stops the batch."""
    if policy not in DUPLICATE_POLICIES:
        raise ValueError(f"unknown duplicate policy {policy!r}; use one of {DUPLICATE_POLICIES}")
    first_row: dict[str, int] = {}
    kept: list[tuple[int, SupportCase]] = []
    for row_number, case in rows:
        if case.case_id in first_row:
            if policy == "reject_batch":
                raise RuleFailure(
                    f"case_id {case.case_id} appears on rows {first_row[case.case_id]} "
                    f"and {row_number}; duplicate_policy is reject_batch"
                )
            ledger.reject(
                row_number,
                RejectCategory.DUPLICATE,
                f"{case.case_id},{case.order_id}",
                f"case_id {case.case_id} already appears on row {first_row[case.case_id]}; "
                "first_wins keeps that row",
            )
            continue
        first_row[case.case_id] = row_number
        kept.append((row_number, case))
    ledger.accepted = len(kept)
    return kept
```

There are exactly two policies, chosen in `config/settings.toml`. `first_wins` keeps the first row and quarantines the repeat, naming the row it clashed with. `reject_batch` stops the whole batch with a `RuleFailure`, because a repeated key means someone must look at the source first.

Real-world example: two payments with the same reference

If a payroll file holds the same employee twice, paying both lines pays one person two salaries, so the bank stops the file. If a newsletter list holds the same email twice, keeping the first copy is harmless. What a repeat would cost decides the policy.

Step 5: the rerun rule

src/meridian_workbench/pipeline.py (start of run_batch)

```
    batch_id = batch_id_for(files, settings.batch_date, json.dumps(rules, sort_keys=True))
    out_dir = settings.output_dir / "batches" / batch_id
    attempt = new_attempt_id()
    summary_path = out_dir / "summary.json"

    if (
        summary_path.is_file()
    ):  # the marker is written last, so its presence means the batch finished
        log_attempt(out_dir, attempt, "reused finished batch; no work repeated")
        return json.loads(summary_path.read_text(encoding="utf-8"))
```

`summary.json` is written last, so if it exists the batch finished. The run then adds one line to the log and returns the saved summary. No rows are parsed again, and no model is called. If there is no summary, the batch is run from the start: a halted batch has no trusted half-results to resume from.

Do: run it twice (3 minutes)

Paste and run (PowerShell: Get-Content instead of cat)

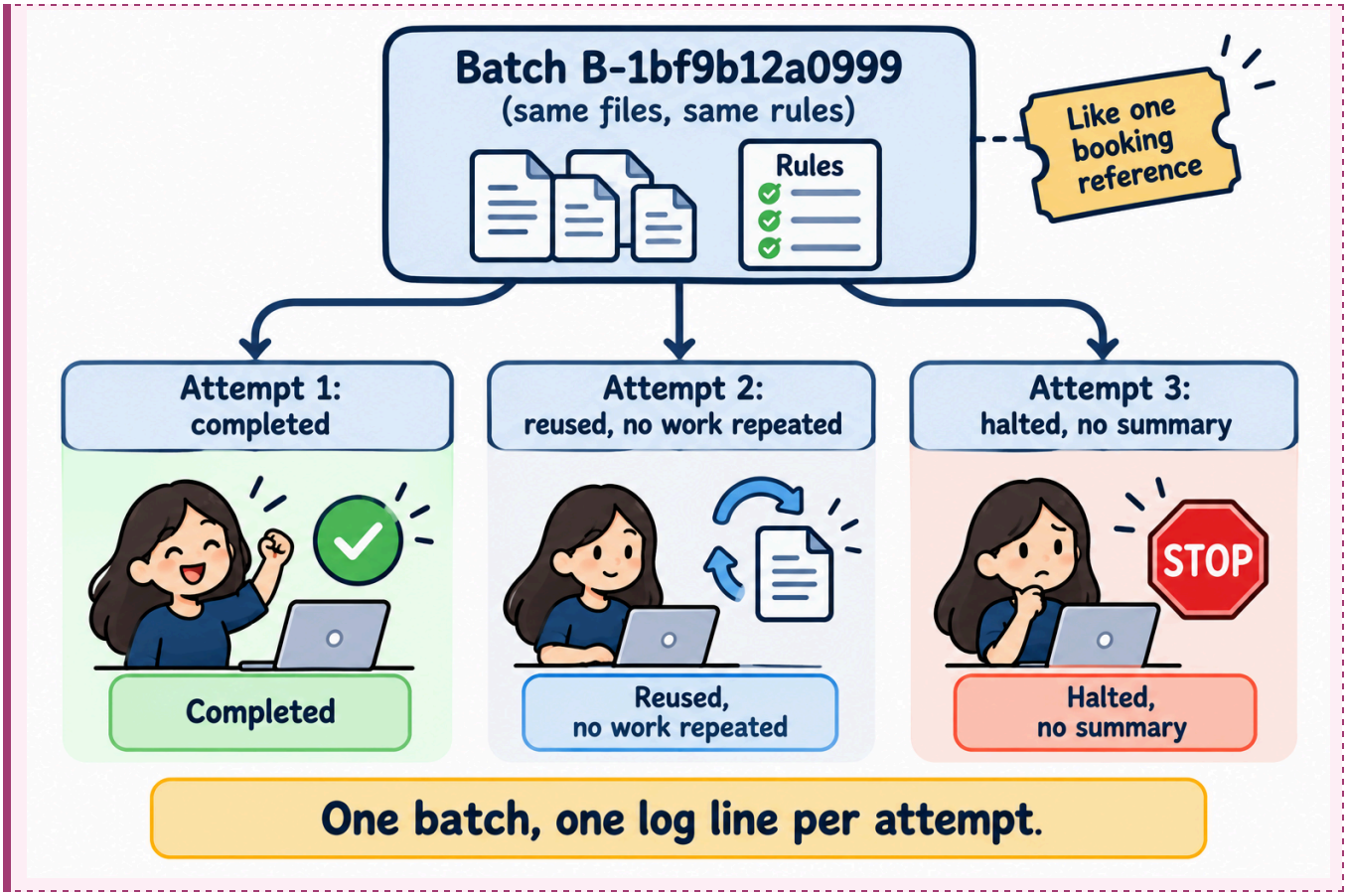
```
python -m meridian_workbench
python -m meridian_workbench
cat outputs/batches/B-1bf9b12a0999/attempts.log
```

What each run prints (the two runs print the same report)

```
cases.csv: input 7, accepted 7, quarantined 0, reconciled yes
notes.json: input 3, accepted 3, quarantined 0, reconciled yes
shipments.csv: input 5, accepted 5, quarantined 0, reconciled yes
inventory.csv: input 7, accepted 7, quarantined 0, reconciled yes
C-1002: OVERDUE_NOT_CLOSED (3 days)
    suggestion (recorded): ADDRESS_CONFIRMATION, review_required=False
C-1004: OVERDUE_NOT_CLOSED (2 days)
C-1005: OVERDUE_NOT_CLOSED (4 days)
     suggestion (recorded): ABSTAINED - The note refers to an earlier message and says nothing about the
issue.
C-1006: OVERDUE_NOT_CLOSED (2 days)
    suggestion (recorded): FAILED_DELIVERY_ATTEMPT, review_required=True
4 of 7 cases need attention
batch B-1bf9b12a0999 (attempt <attempt-id>)
```

attempts.log: two attempts (the report always prints the attempt that finished)

```
<time>  <attempt-id>  completed
<time>  <attempt-id>  reused finished batch; no work repeated
```



Pause and predict

Ask: “A batch halted. Someone fixes line 3 of cases.csv. What happens on the next run?” The file’s bytes changed, so the batch ID changes and the run uses a new folder. The old folder stays as the record of the halt.

Check for understanding

Ask: After a batch has completed, how many model calls does a rerun make?

Real-world example: A restaurant that has already served order 42 does not cook it again because the waiter reads the ticket twice.

Answer: None. The rerun finds summary.json, logs the attempt and returns the saved summary.

Common mistakes to expect

- Using the time or a random number as the batch ID, so every run looks like a new batch.
- Writing the summary first. A crash after that leaves a “finished” marker on unfinished work.
- Leaving the duplicate rule unwritten, so it depends on whoever wrote the loader.
- Deleting the outputs folder to “fix” a rerun. That throws away the record of what already happened.

Quiz 4 of 6 | After Section 5 (minute 67) | Multiple choice (one answer) | about 90 seconds

Checks: What makes a batch ID change.

Question. Which change gives a different batch ID?

A. Running the same files on another laptop

B. Changing ai.mode from recorded to live

C. Listing the four files in a different order

D. Copying the data folder to a new location

Correct answer: B

Why B is correct: The AI mode is part of the rules text inside the fingerprint, so a live run gets its own batch.

Why the others are wrong: A, C and D keep the same file names, file bytes, date and rules, so they give the same ID. The files are sorted by name before hashing, which is why order does not matter. That sameness is what makes the ID useful.

Instructor tip: Show the two-line output from Step 1: recorded and live, same files, two IDs.

Transition

A batch can now be named, recorded and rerun safely. Next: what to do when a call to an outside service fails, and how to try again without making things worse.

Section 6: Bounded retry: backoff, jitter and tenacity

Duration	14 minutes (minute 67 to 81), including Quiz 5
Teaching objective	Learners decide which errors are worth retrying, write a retry loop with a limit and a growing random wait, and explain why retries must live in one place only.

Say

Some failures fix themselves. A server is busy for a moment, a network packet is lost. If we wait and try again, it works. But a retry that is too fast, or repeated forever, adds load to a service that is already struggling. A good retry needs three things: a rule for which errors to retry, a limit, and a wait that grows.

Real-world example: calling a friend

If your friend's phone is busy, you call back in a minute, then in five minutes, then you send a message and stop. If the number does not exist, calling again will never help. Good retry code makes the same two decisions.

Step 1: which errors are worth retrying

Error	Retry?	Why
LlmTimeout	Yes	No answer in time. A later try may get through.
LlmRateLimited (429)	Yes	The service asked us to slow down. Waiting is what it asked for.
LlmServerError (5xx)	Yes	A failure on their side, often brief.
MissingApiKeyError	No	Waiting will not add a key.
LlmCallFailed, or a bad reply	No	Unknown or repeatable. The same request fails the same way.

Step 2: backoff and jitter (read together)

Open `src/meridian_workbench/retry.py`, pasted in Section 4. Start with the wait.

src/meridian_workbench/retry.py (the rule and the wait)

```
RETRYABLE: tuple[type[Exception], ...] = (LlmTimeout, LlmRateLimited, LlmServerError)


def backoff_delay(attempt: int, base: float, cap: float, rng: random.Random) -> float:
    """A random wait from 0 up to the current ceiling. The ceiling doubles, up to the cap."""
    ceiling = min(cap, base * 2 ** (attempt - 1))
    return rng.uniform(0, ceiling)
```

The ceiling is the longest wait allowed for an attempt. It starts at `base` and doubles each time, until it hits `cap`. Settings use a base of 0.5 seconds and a cap of 8 seconds. The real wait is a random number between zero and the ceiling. That random part is called full jitter.

Ceiling for attempts 1 to 6 (attempt, seconds)

```
1 0.5
2 1.0
3 2.0
4 4.0
5 8.0
6 8.0
```

> **Real-world example: the school bell**
>
> When one bell rings, 800 students reach the gate at once and it jams. If classes leave a minute or two apart, the same students pass easily. Random waits spread retries the same way, so clients that failed together do not all come back at the same second.

Step 3: the loop (read together)

src/meridian_workbench/retry.py (call_with_retry)

```
def call_with_retry(
    operation: Callable[[], T],
    *,
    max_attempts: int,
    base: float,
    cap: float,
    rng: random.Random,
    sleep: Callable[[float], None] = time.sleep,
) -> T:
    if max_attempts < 1:
        raise ValueError("max_attempts must be at least 1")
    for attempt in range(1, max_attempts + 1):
        try:
            return operation()
        except RETRYABLE as error:
            if attempt == max_attempts:
                raise  # out of attempts: pass the original error up, with its traceback
            delay = backoff_delay(attempt, base, cap, rng)
            print(f"attempt {attempt} failed ({type(error).__name__}); waiting {delay:.2f}s")
            sleep(delay)
    raise AssertionError("unreachable: the loop always returns or raises")
```

What each piece does

- `max_attempts` is the limit, counting the first try. A loop with no limit can hang a batch forever.
- `except RETRYABLE` catches only the three temporary errors. Anything else passes straight up, untouched.
- On the last attempt, a bare `raise` passes the original error up with its traceback.
- `sleep` and `rng` are passed in, so a test can skip the real waiting and get the same random numbers every time.

Do: the retry drill (2 minutes)

Paste into python, from the project root

```
import random
from meridian_workbench.llm_client import LlmTimeout
from meridian_workbench.retry import call_with_retry

calls = []

def flaky():
    calls.append(1)
    if len(calls) < 3:
        raise LlmTimeout('no reply within the timeout')
    return 'ok'

print('result:', call_with_retry(flaky, max_attempts=3, base=0.5, cap=8.0, rng=random.Random(4),
sleep=lambda s: None))
```

```
print('calls:', len(calls))
```

What Python prints

```
attempt 1 failed (LlmTimeout); waiting 0.12s
attempt 2 failed (LlmTimeout); waiting 0.10s
result: ok
calls: 3
```

Two timeouts, two short random waits, then success. The seed makes the waits repeatable; the lambda skips the real pause.

Step 4: tenacity, and the danger of stacked retries

Many teams use a library called tenacity, which does the same job with a decorator. The code below is for recognition only: it is not in the project, not installed and not run.

Illustrative only: not in the project

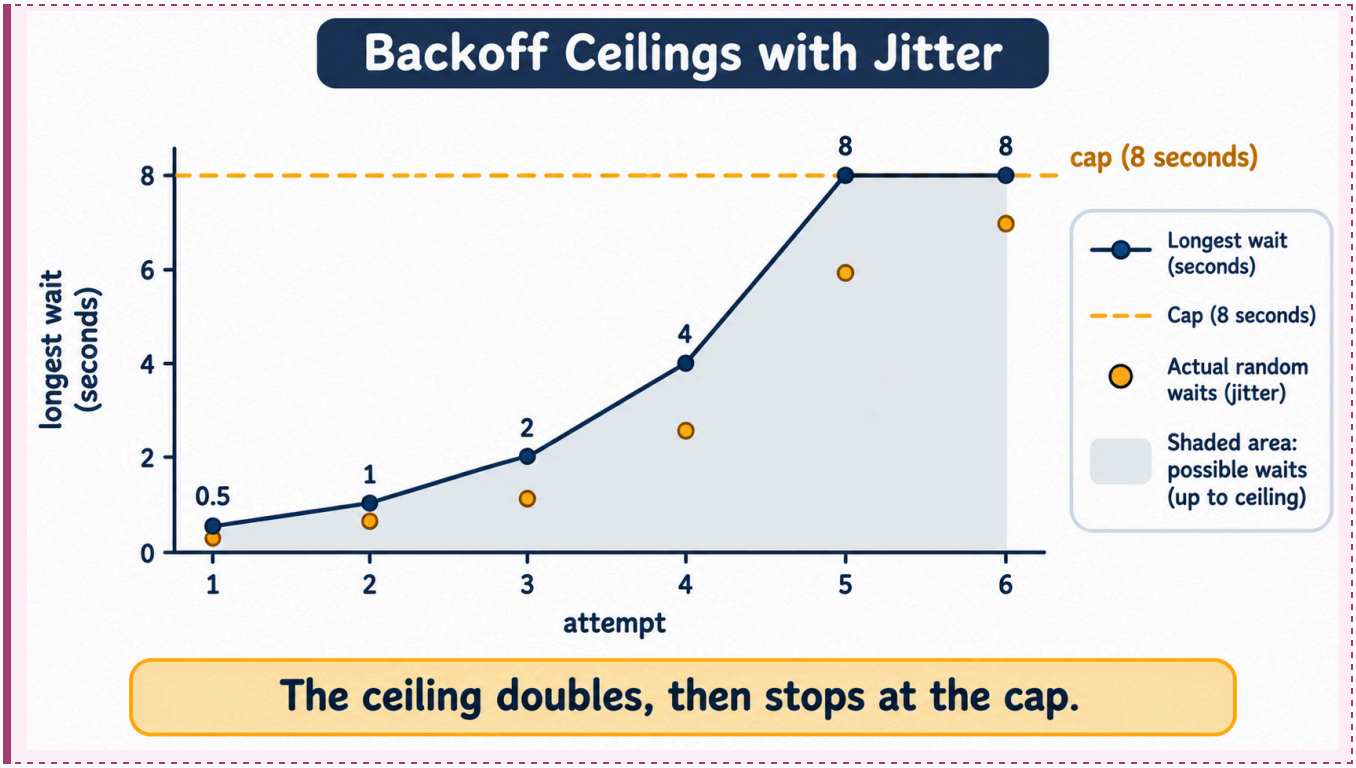
```
from tenacity import retry, retry_if_exception_type, stop_after_attempt, wait_random_exponential

@retry(
    retry=retry_if_exception_type((LlmTimeout, LlmRateLimited, LlmServerError)),
    stop=stop_after_attempt(3),
    wait=wait_random_exponential(multiplier=0.5, max=8),
)
def call_model_with_tenacity(prompt, settings):
    ...
```

Tenacity saves lines; our loop keeps every rule visible in one file. Either is fine. Two retry layers in one path are never fine: three SDK retries inside our three attempts turn one failing request into nine calls. So `llm_client.py` switches the SDK's own retry off:

src/meridian_workbench/llm_client.py (one line)

```
        max_retries=0,  # our own retry in retry.py decides; the SDK must not add its own
```



Check for understanding

Ask: Three layers of code each retry three times. How many calls reach the service for one failing request?

Real-world example: Three people each ring the same doorbell three times: the person inside hears nine rings.

Answer: Twenty-seven (3 × 3 × 3). With one retry layer and three attempts, it is three.

Common gotcha

A missing key is never retried. Waiting cannot create a key, and retrying would only make a broken setup slower to report.

Quiz 5 of 6 | After Section 6 (minute 81) | Multiple choice (one answer) | about 90 seconds

Checks: Which errors are worth retrying.

Question. Which error does call_with_retry never retry?

A. LlmTimeout, when the call takes too long

B. LlmRateLimited, when there are too many calls

C. LlmServerError, when the service has a fault

D. MissingApiKeyError, when no key is set

Correct answer: D

Why D is correct: A missing key will still be missing after any wait, so retrying cannot succeed.

Why the others are wrong: A, B and C are temporary problems on the network or the service side, so a later try can work. That is why they are the three errors in the retry list.

Instructor tip: Ask learners to say “could a later try succeed?” for each option out loud.

Transition

Retries now wait sensibly and stop at a limit. Next: what the report says when the retries run out, or when a model reply is unusable. The rule is simple: never invent an answer.

Section 7: Model failures, statuses, the cache and the golden set

Duration	17 minutes (minute 81 to 98), including Quiz 6
Teaching objective	Learners turn every model outcome into one readable status, keep the rule facts when the model is unavailable, reuse only good answers, and extend the golden set with an ambiguous and an insufficient case.

Say

A model is an outside service. It can be slow, it can say we asked too often, and it can answer with something we cannot use. Each of these becomes one status a person can read. And we never fill the gap with an invented answer. The facts from our rules come through whatever the model does.

Real-world example: the weather on the morning news

If the weather feed is down, the newsreader says “forecast unavailable this morning”. She does not read out yesterday’s forecast as if it were today’s, and the traffic and headlines still go out on time.

Step 1: one call, one name for each failure (read together)

Open `llm_client.py`, pasted in Section 4, and read the part that catches errors.

src/meridian_workbench/llm_client.py (the call and its errors)

```
    try:
        response = client.messages.create(
            model=settings.model,
```

```
            max_tokens=settings.max_tokens,
            messages=[{"role": "user", "content": prompt}],
        )
    except anthropic.APITimeoutError as error:  # first: a timeout is also an APIError
        raise LlmTimeout("no reply within the timeout") from error
    except anthropic.RateLimitError as error:
        raise LlmRateLimited("rate limit reached (429)") from error
    except anthropic.InternalServerError as error:
        raise LlmServerError(f"server error ({error.status_code})") from error
    except anthropic.APIError as error:  # last: the catch-all
        # Name the kind of failure only. Never put the key or the request in the message.
        raise LlmCallFailed(type(error).__name__) from error
```

- The timeout is caught first, because the SDK's timeout is also a kind of `APIError`. The catch-all must come last.
- Each SDK error becomes one of our names, so the rest of the program never imports the SDK.
- The message names only the kind of failure. A key or a customer's note must never appear in an error message, because messages are copied into logs and tickets.

Step 2: four statuses (read together)

Open `ai_client.py`. Every outcome ends in exactly one status.

Status	When	What the person sees
OK	Valid JSON with a known issue family	The family, the summary and the quoted evidence
ABSTAINED	The model said the note does not say what the problem is	The model's reason
INVALID_REPLY	Not JSON, no issue_family field, an unknown family, or a wrong field type	The exact problem
UNAVAILABLE	Timeout, rate limit or server error after all retries, another failure, or no key	The reason; no suggestion is invented

Here is `interpret_reply` on four replies: a good one, an honest abstention, plain text, and valid JSON with the wrong field name.

What Python prints

```
OK | ADDRESS_CONFIRMATION
ABSTAINED | Note says nothing about the problem.
INVALID_REPLY | the reply was not valid JSON
INVALID_REPLY | the reply has no issue_family field
```

The fourth reply looks like an answer, but it has no `issue_family` field, so it is refused. Recorded answers go through the same function, so recorded and live replies are always judged the same way.

Step 3: the quote must be in the note

src/meridian_workbench/ai_client.py (the grounding check)

```
    warnings: list[str] = []
    if span is None:
        warnings.append("no evidence_span was quoted")
    elif span not in note.text:
        warnings.append("evidence_span does not appear in the note")
    review = bool(data.get("review_required", True)) or bool(warnings)
```

The model must quote the words that support its answer, and the code checks that the quote really is in the note. Golden item G-09 shows why. The note says only "Customer asking about parcel", but the model quoted "parcel delayed".

What Python prints

```
note: Customer asking about parcel
status: OK
```

```
family: CARRIER_DELAY
evidence: parcel delayed
review_required: True
warnings: ('evidence_span does not appear in the note',)
```

The answer is kept, but it is flagged and review is forced on. Nobody should have to trust a quote that cannot be found in the evidence.

Step 4: no invented fallback

src/meridian_workbench/ai_client.py (when the retries run out)

```
    except LlmTimeout:
        return _unavailable(
            case_id, note, settings, f"model timed out after {settings.retry_max_attempts} attempts"
        )
    except LlmRateLimited:
        return _unavailable(
            case_id, note, settings, f"rate limited after {settings.retry_max_attempts} attempts"
        )
    except LlmError as error:
        return _unavailable(case_id, note, settings, str(error))
```

In Class 3, a failed live call fell back to the recorded answer. That was convenient, and wrong: recorded answers are test data, not a backup for a real service. Now a failed call is UNAVAILABLE with a reason. Try it in live mode with no key:

Paste and run (PowerShell: set $env:MERIDIAN_AI_MODE first)

```
MERIDIAN_AI_MODE=live python -m meridian_workbench
```

What Python prints

```
cases.csv: input 7, accepted 7, quarantined 0, reconciled yes
notes.json: input 3, accepted 3, quarantined 0, reconciled yes
shipments.csv: input 5, accepted 5, quarantined 0, reconciled yes
inventory.csv: input 7, accepted 7, quarantined 0, reconciled yes
C-1002: OVERDUE_NOT_CLOSED (3 days)
    suggestion (live): UNAVAILABLE - MERIDIAN_LLM_API_KEY is not set
C-1004: OVERDUE_NOT_CLOSED (2 days)
C-1005: OVERDUE_NOT_CLOSED (4 days)
    suggestion (live): UNAVAILABLE - MERIDIAN_LLM_API_KEY is not set
C-1006: OVERDUE_NOT_CLOSED (2 days)
    suggestion (live): UNAVAILABLE - MERIDIAN_LLM_API_KEY is not set
4 of 7 cases need attention
batch B-5ab2e0c27bc3 (attempt <attempt-id>)
```

Every flagged case keeps its facts and days overdue; only the suggestion says UNAVAILABLE. The batch ID differs from the recorded run, as Section 5 promised.

Step 5: the cache: reuse good answers, never failures

src/meridian_workbench/ai_client.py (the cache lookup)

```
    key = f"{note.note_id}:{settings.model}:{settings.prompt_version}"
    cache = load_cache(settings.cache_path)
    if key in cache:
        meta = CallMetadata(
            "cached",
            settings.model,
            settings.prompt_version,
            0,
            0,
            "cache",
            "reused from an earlier successful run",
        )
        return interpret_reply(cache[key], case_id, note, meta)
```

A live call costs time and money, so each usable answer is kept under a key made of the note ID, the model and the prompt version. A new model or prompt means a new key. Only OK and ABSTAINED replies are stored: a cached timeout would be replayed as if it were a fact.

outputs/suggestion_cache.json after one good live reply

```
{
    "N-01:claude-haiku-5-5:v1":  "{\"issue_family\":  \"ADDRESS_CONFIRMATION\",  \"summary\":  \"Held  at depot.\", \"evidence_span\": \"awaiting address confirmation\", \"review_required\": false}"
}
```

Step 6: two new golden cases

The golden set is a small set of notes with answers written down before the model runs. Copy the two items from `golden/extra_cases.json` into `golden/golden_set.json` (or paste the final file), and add their two rows to `golden/scoring_baseline.csv`, and paste `golden.py`, which now prints each status.

Item	Kind	Note	Expected
G-11	AMBIGUOUS	Parcel still not here. Last scan Monday at depot, customer says they were home all week.	CARRIER_DELAY or FAILED_DELIVERY_ATTEMPT
G-12	INSUFFICIENT	Call dropped after the greeting. Customer name only.	Abstain: no family

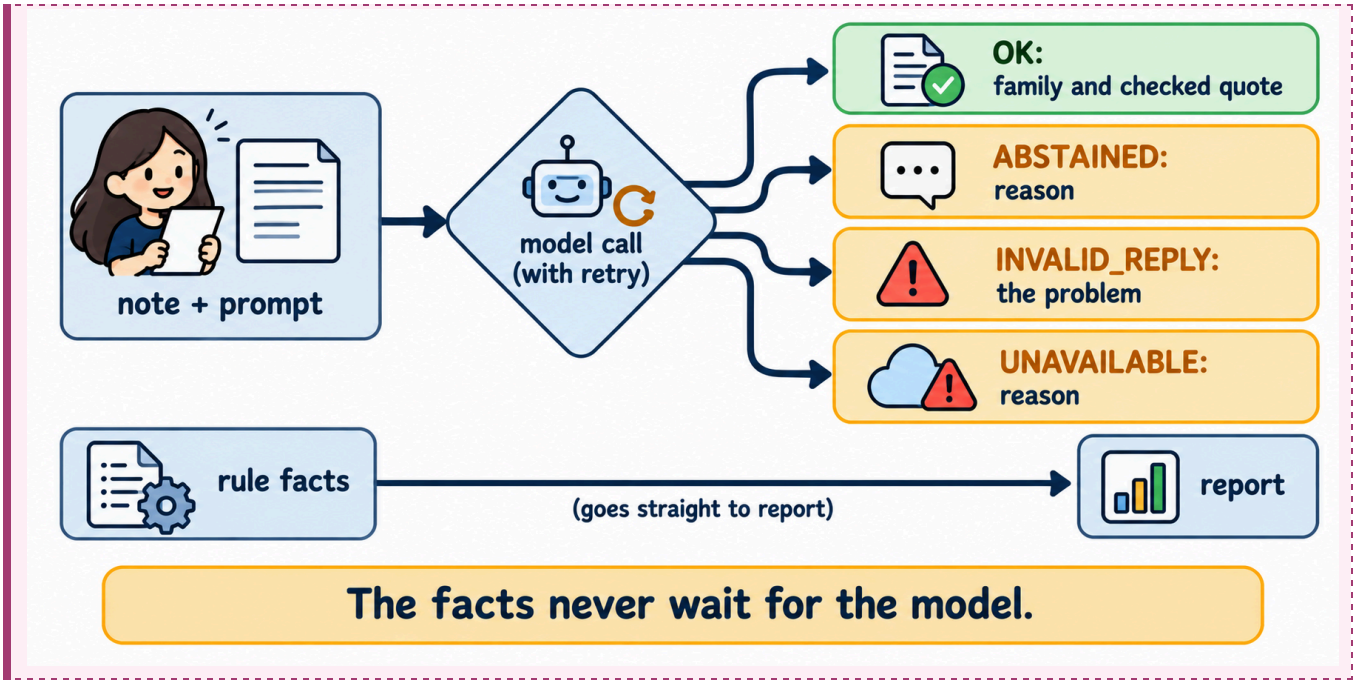
Paste and run

```
python -m meridian_workbench.golden run
```

What Python prints

```
G-01  CLEAR                OK  ADDRESS_CONFIRMATION  |  evidence:  'awaiting  address  confirmation'  |  review_required=False | recorded
G-02 CLEAR        OK FAILED_DELIVERY_ATTEMPT | evidence: 'cust n/a x2, rtn to dpt' | review_required=True | recorded
G-03 ABSTAIN      ABSTAINED  | evidence: None | review_required=True | recorded
G-04 RULE_ONLY    (rule closes this case, no model call)
G-05 RULE_ONLY    (rule closes this case, no model call)
G-06 CLEAR        OK STOCK_SHORTAGE | evidence: 'out of stock' | review_required=False | recorded
G-07 CLEAR        OK CARRIER_DELAY | evidence: 'delay at sorting hub' | review_required=False | recorded
G-08 AMBIGUOUS     OK  FAILED_DELIVERY_ATTEMPT  |  evidence:  'cust  unreachable'  |  review_required=False  |  recorded
G-09 INSUFFICIENT OK CARRIER_DELAY | evidence: 'parcel delayed' | review_required=True | recorded
G-10 CLEAR        OK DAMAGED_IN_TRANSIT | evidence: 'box crushed' | review_required=True | recorded
G-11 AMBIGUOUS      OK  CARRIER_DELAY  |  evidence:  'Last  scan  Monday  at  depot'  |  review_required=True  |  recorded
G-12 INSUFFICIENT ABSTAINED  | evidence: None | review_required=True | recorded
```

G-11 gets CARRIER_DELAY with review required, which is one of its two acceptable answers. G-12 abstains, as it should. The runner does not grade anything. People do, with the tool's `blank-sheet` and `score` commands, because only a person can judge whether a summary claims something the note never said.



Check for understanding

Ask: The model times out on case C-1005 in live mode. What does the report show?

Real-world example: The weather feed is down: the bus timetable is still printed, and the forecast line says "unavailable".

Answer: C-1005 is still flagged with its facts and days overdue. The suggestion shows UNAVAILABLE and the reason. Nothing is guessed in its place.

Common mistakes to expect

- Falling back to a recorded answer when a live call fails.
- Caching every reply, including timeouts and invalid replies.
- Catching the general `APIError` before `APITimeoutError`, so timeouts are never recognised.
- Putting the key, the prompt or the note text into an error message.

Quiz 6 of 6 | After Section 7 (minute 98) | Multiple choice (one answer) | about 90 seconds

Checks: What the report shows when the model is unavailable.

Question. In live mode, the model times out on all three attempts for C-1002. What does the report show for C-1002?

A. The flagged case, with UNAVAILABLE and the reason

B. The recorded answer for C-1002, marked recorded

C. Nothing, as the case is removed from the report

D. The most common issue family, marked for review

Correct answer: A

Why A is correct: The rule facts do not depend on the model, so the case stays flagged. The suggestion gets a status and a reason (UNAVAILABLE - model timed out after 3 attempts), never an invented answer.

Why the others are wrong: B uses test data as if it were a live answer. C loses a flagged case that a person needs to see. D is a guess dressed up as an answer.

Instructor tip: Point back at the live run with no key: every flagged case is still there.

Transition

The model is now a careful assistant: it answers with a status, its quotes are checked, and only good answers are reused. Next we prove that every failure path really behaves this way.

Section 8: pytest checks for the failure paths

Duration	12 minutes (minute 98 to 110)
Teaching objective	Learners read eight named checks as promises, finish the stub bodies in the starter pack, and watch each one turn green.

> **Say**
> *A promise that is not tested is only a hope. Each failure path from today has a name in plain words, and each name is a check. If a future change quietly breaks a promise, a check turns red before anyone sees a wrong report.*

Step 1: the eight checks

Check	The promise	Section
test_malformed_date_is_quarantined	07/03/2025 is never guessed; the row is quarantined with its raw text	3
test_impossible_date_is_quarantined	2025-02-30 is refused as “not a real calendar date”	2, 3
test_duplicate_case_id_first_wins	A repeated case ID keeps the first row; the repeat is a DUPLICATE	5
test_missing_required_source_stops_run	A missing file stops the run before any file is read	4
test_reconciliation_balances	The books balance, and a mismatch raises an error	4
test_timeout_is_unavailable_and_keeps_facts	A timeout gives UNAVAILABLE and the facts stay	7
test_rate_limit_is_retried_then_unavailable	A 429 is retried up to the limit, then UNAVAILABLE	6, 7
test_rerun_same_batch_does_not_duplicate	A rerun makes no new model calls	5

Seven stubs are in `tests/test_failures.py` and one is in `tests/test_rerun.py`. Each has a docstring that states the promise and says what to assert. Two helpers are ready in `tests/conftest.py`: `settings_for` gives settings that write into a temporary folder, use live mode and carry a fake key; `data_dir` gives a private copy of the data, so no test can change the real files.

Step 2: read one finished check

tests/test_failures.py (one finished check)

```
def test_malformed_date_is_quarantined(tmp_path: Path) -> None:
    path = write_cases(tmp_path / "cases.csv", "C-1,O-1,SHIPPED,07/03/2025")
    rows, ledger = load_cases(path)
    assert rows == []
    assert ledger.accepted == 0
    assert len(ledger.rejected) == 1
    rejected = ledger.rejected[0]
    assert rejected.category is RejectCategory.MALFORMED
    assert rejected.row_number == 2
    assert "07/03/2025" in rejected.raw
    assert "YYYY-MM-DD" in rejected.reason
```

`tmp_path` is a fresh empty folder that pytest gives each test. The check writes a one-row file there and asserts every part of the promise: no case accepted, one rejection, the category, the line number, the raw text and the reason. A check that only asserted the category could pass for the wrong reason.

Step 3: the rerun check, which counts model calls

tests/test_rerun.py

```
def test_rerun_same_batch_does_not_duplicate(
    settings_for: Callable[..., Settings], data_dir: Path
) -> None:
    calls: list[str] = []

    def counting(prompt: str, settings: Settings) -> LlmReply:
        calls.append(prompt)
        return LlmReply(REPLY, "test-model", 10, 5, "end_turn")

    settings = settings_for()
    first = run_batch(data_dir, settings, caller=counting)
    calls_after_first = len(calls)
    assert calls_after_first == 3  # three flagged cases have a note

    second = run_batch(data_dir, settings, caller=counting)
    assert len(calls) == calls_after_first  # no new model calls on the rerun
    assert second["batch_id"] == first["batch_id"]
    batches = list((settings.output_dir / "batches").iterdir())
    assert len(batches) == 1  # still one folder for one batch

    log = (batches[0] / "attempts.log").read_text(encoding="utf-8").splitlines()
    assert len(log) == 2
    assert log[1].endswith("reused finished batch; no work repeated")
```

`counting` is a fake model: it records each prompt and returns a fixed reply. The first run makes three calls, one for each flagged case with a note. The second run must make none, keep the same batch ID and folder, and add exactly one log line.

Do: finish the stubs (5 minutes)

Paste and run

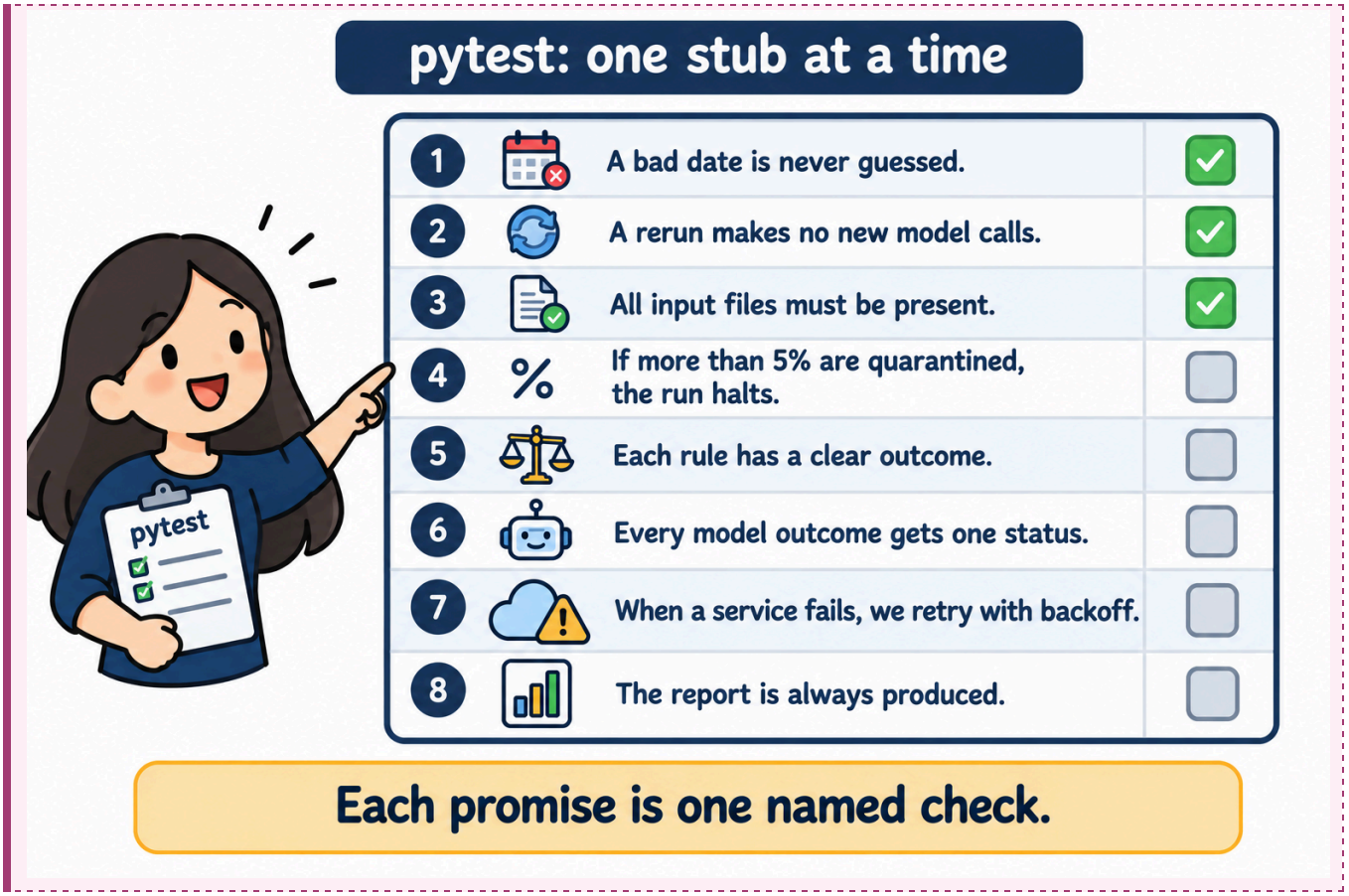
```
python -m pytest -q tests/test_failures.py tests/test_rerun.py
python -m pytest -q
```

In the untouched starter pack, the full run ends with `8 failed, 26 passed`: the eight stubs. Paste the updated `test_ai_client.py`, `test_domain.py`, `test_golden.py`, `test_loading.py` and `test_rules.py` and the new `test_parsing.py` and `test_retry.py` from the Final directory. Then finish the stubs, starting with `test_malformed_date_is_quarantined`. When a check stays red, read the lines starting with `E`: pytest prints what it expected and what it got. The finished repository gives:

What pytest prints in the final repository

```
46 passed
```

> **Pause and predict**
>
> Ask: “A check passed on the first try. How do you know it tests anything?” Break the code on purpose, for example let 07/03/2025 through `parse_date`, and run it again. It must go red.



Check for understanding

Ask: Why does the timeout check use a fake caller instead of the real model?

Real-world example: A fire drill uses a pretend alarm. Nobody sets the building on fire to test the exits.

Answer: A fake caller can fail exactly when we want, every time, with no key, no internet and no cost. A real outage cannot be ordered on demand.

Common mistakes to expect

- Asserting only the category, not the line number, raw text and reason.
- Writing test files into `data/` instead of `tmp_path`, which changes the real data.
- Letting a retry test really sleep. Pass `sleep=lambda s: None`.
- Leaving a real API key in the environment while testing.

If learners struggle

Learners who are behind can paste the finished bodies from `tests/test_failures.py` and `tests/test_rerun.py` in the Final directory, then make one of them fail on purpose and fix it.

Transition

Every promise is now a check that runs on every change. Last, we put the batch into words for the people who depend on it.

Section 9: The incident update and close

Duration	10 minutes (minute 110 to 120)
Teaching objective	Learners write an incident-style update from real numbers, keep facts apart from guesses, name the action needed, and close with a green run committed and pushed.

Say

Back to the ops lead's question: did every case get checked, and can I trust the number? An update answers it with facts. It is not a story. It says what happened, what it affects, and who must do what next.

Real-world example: the pilot's announcement

"We are delayed by 40 minutes because of a technical check on the left engine. You can stay seated. I will update you at 10:30." Facts, impact, action, next update. No guesses about the cause.

Step 1: the template

`docs/incident_update_template.md` came with the starter pack. Its headings, in order:

Heading	What goes there
Summary and Status	One or two sentences; Halted, Degraded or Complete
Batch	Batch ID, batch date and attempt ID from attempts.log
Facts	What the logs and files show, and the exact message
Affected counts	Input, accepted, quarantined, suggestions, published
Operational impact	What people cannot do today, and what they still can
Required action	Owner, action, by when
Not yet known	Open questions; any guess labelled as a guess
Next update	A time, or the event that triggers it

Step 2: write the update for the halted batch

Every number below comes from the halted run in Section 4.

Example update (copy the shape, not the words)

```
# Incident update: B-a729b8cd1811

Summary: The batch for 2025-03-10 halted. No report was published.
Status: Halted
Batch: B-a729b8cd1811 for 2025-03-10. Attempt: from attempts.log

Facts:
- Halt message: cases.csv: 4 of 8 rows (50%) were quarantined; the limit is 5%.
- Quarantined: line 3 (status RETURNED), line 4 (date 07/03/2025), line 6 (duplicate C-3004),
  line 7 (date 2025-02-30).

Affected counts:
- cases.csv: 8 input, 4 accepted, 4 quarantined.
- Suggestions: 0 (no model call was made). Published: 0.

Operational impact: No attention report for this batch. Teams must use the previous report and say so.

Required action: Owner: data owner for cases.csv. Fix or confirm the four rows, then run a new batch.
Do not rerun before that: the same file gives the same halt.

Not yet known: whether RETURNED is a new status we should accept. This is a question, not a guess.

Next update: when the data owner replies.
```

Notice what it leaves out: no guess about why the data changed, and no promise of a time it cannot keep.

Before you send an update

- Every number comes from a file or a run, and you can say which.
- The batch ID and attempt ID match summary.json or attempts.log.
- Every guess sits under “Not yet known” and is labelled as a guess.
- No key, no note text and no personal data. Counts and IDs are enough.



Check for understanding

Ask: The update says “the supplier probably changed their export”. Where does that sentence belong?

Real-world example: A doctor writes “possible sprain, X-ray pending”, not “sprained ankle”, until the X-ray comes back.

Answer: Under “Not yet known”, labelled as a guess. Nobody has confirmed it yet.

Step 3: close: a green run, committed and pushed

Paste the updated `README.md`, `docs/terminology.md`, `pyproject.toml` and `.env.example` from the Final directory, then run the three tools.

Paste and run

```
python -m pytest -q
python -m mypy
python -m ruff check .
```

What the three tools print

```
46 passed
Success: no issues found in 27 source files
All checks passed!
```

Save and upload

```
git status
git add -A
git commit -m "Add intake gates, batch identity, safe reruns and failure checks"
git push
```

What we built today

- Every source row is accepted, or quarantined with a category, a reason and its raw text.
- Four kinds of failure, each with its own outcome, and a halt before any model call when the batch is not good enough.
- A stable batch ID, an attempt ID for each run, and a rerun that returns the finished result.
- Bounded retries with a cap and jitter, for temporary errors only.
- Model answers with four statuses, checked quotes, a cache of good answers only, and twelve golden cases.
- Eight named checks, and an update the ops lead can act on.

Say

Monday's question was "did every case get checked?" Today the program can answer it with a count, a quarantine file and a log. That is the difference between a run that finished and a run you can trust.